Harnessing Web Accessibility Tools for WCAG 2.1 Migration of a Design System

Conference Paper · December 2023

DOI: 10.1109/CoNTESA61248.2023.10384953

Hardik Shah
Rochester Institute of Technology

Harnessing Web Accessibility Tools for WCAG 2.1 Migration of a Design System

Hardik Shah
Department of Information Technology
Rochester Institute of Technology
USA
hds6825@rit.edu

2023 International Conference on Computing, Networking, Telecommunications & Engineering Sciences Applications (CoNTESA) | 979-8-3503-8275-4/23/$31.00 ©2023 IEEE | DOI: 10.1109/CoNTESA61248.2023.10384953

***Abstract*— This paper addresses the critical challenges of upgrading Design Systems to conform to the Web Content Accessibility Guidelines (WCAG) 2.1, a key step in fostering digital inclusivity and maintaining a competitive edge in today's business landscape. It scrutinizes the intricacies involved in the transition from WCAG 2.0 to WCAG 2.1, focusing on the integration of various web accessibility tools such as screen readers, color contrast checkers, and automated testing frameworks within the development workflow. This study methodically evaluates these tools' effectiveness in identifying and remedying accessibility flaws during the migration process. Additionally, the paper acknowledges the initial disruptive potential of incorporating these technologies into established development practices and the necessity of training developers and designers to adeptly handle these changes. The research culminates in highlighting the significant outcomes: by embedding accessibility considerations into the development cycle, not only do businesses achieve compliance with legal standards, but they also enhance user experience, thereby creating a more inclusive and user-friendly digital environment. This transition, despite its initial challenges, is shown to be a vital investment for businesses striving for excellence in the digital era.**

Keywords*— *Web Accessibility Tools, WCAG 2.1 Migration, Design Systems, Web Accessibility Testing, Web Accessibility

I. Introduction

In an increasingly dynamic environment for Digital Interfaces, Design Systems allow for uniformity, expandability and usability on web-based software programs. Upgrading these systems towards Web Content Accessibility Guidelines (WCAG) 2.1 exemplifies efforts for inclusivity, while also being legally compliant and offering enhanced experiences to people with disabilities. Migration process entails an in-depth assessment of existing accessibility benchmarks against WCAG 2.1 stipulations. In this regard, nothing can be more critical than web accessibility tools that automatically assess compliance, identify deficiencies and provide recommendations on their addressing.

Tools such as screen readers, color contrast checkers, and automated accessibility testing frameworks serve as both a measure and a beacon—quantifying current accessibility levels and guiding the enhancement process. These tools assist in validating semantic structure, ensuring keyboard navigability, and verifying visual design against the improved color contrast ratios and reflow requirements introduced in WCAG 2.1 [1]. The use of these tools effectively bridges the gap between WCAG 2.0 and 2.1, enabling developers and designers to systematically address and rectify deficiencies.

As the digital domain becomes increasingly inclusive, the onus is on enterprises to adopt these advanced accessibility standards. This paper delves into the integral role of web accessibility tools in facilitating the transition of a Design System from WCAG 2.0 to 2.1, illustrating the nuances of the migration process, and underscoring the importance of these tools in upholding the ethos of digital accessibility.

Considering the crucial role that Design Systems play in contemporary web development, a thorough guide aimed at transferring these foundations might have significant effects on accessibility. With its distinct emphasis on the subtle use of online accessibility tools throughout the migration path, this paper provides comprehensive approaches to assess compliance, find gaps, apply WCAG 2.1 criteria, and maintain standards via CI/CD automation [2]. Since Design Systems form the foundation of many web and mobile apps, study helps firms to make sure these essential elements are in line with inclusive design principles and legally required standards.

II. Literature Review

A substantial database of research emphasizes the critical significance of online accessibility tools in aiding the transformation of Design Systems to WCAG 2.1 criteria. This explains how Design Systems work as the foundation of web application development, highlighting their importance in encapsulating design patterns and implementing accessibility protocols. Given its importance, the article recommends embedding WCAG 2.1 directly into the Design System as a deliberate option to ensure consistency across applications [3]. Other Scholars go into detail into screen readers like JAWS, explaining how they might help developers evaluate keyboard navigability and focus order. This article investigates Windows Narrator's ability to deliver feedback on interactive items such as buttons and links. Similarly, technologies like WAVE, Chrome Lighthouse, and Accessibility Insights aid in automated examinations of semantic structure and markup. Their assessments highlight flaws in the HTML structure, a lack of alt text, and the absence of landmark roles [4]. The WebAIM color contrast checker, as shown by researchers, provides immediate feedback on color combinations, offering alternatives that fulfill WCAG standards.

Scholars recommend post-migration Build and CI automation with aXe and Lighthouse CI to insert accessibility tests directly into processes. Prioritizing accessibility from the start, as proponents of the 'Shift Left' method argue, greatly boosts standards [5]. According to the literature, organizations may simplify the transition to WCAG 2.1 and build a culture of digital inclusion by carefully using these technologies during the migration process.

While previous research has established useful frameworks and best practices for using online accessibility tools throughout the migration process, further study is required to fully understand the benefits of these tools. Research that looks at incorporating new technologies like axe-core and Lighthouse CI into contemporary web development processes will be useful as Design Systems and web frameworks continue to grow [6]. Comparative evaluations of the advantages and disadvantages of various instruments may also provide light on the best uses for each. The Shift Left strategy might be strengthened by longitudinal studies that look at the long-term effects of giving accessibility testing top priority early in the development lifecycle. By use of surveys and interviews with web development teams, qualitative research may provide insights into the difficulties encountered during migration trips and identify potential areas for tool improvement. In the end, targeted studies on making the most of online accessibility technologies will provide the practical knowledge required to develop digital experiences that are inclusive of all users.

III. Importance of Design Systems

Design Systems serve as the backbone of web application development. They offer a unified set of components, patterns, and standards, ensuring consistent user experiences across platforms and devices. Design Systems encapsulate design tokens, icons, UI components, a reference website, best practices, facilitate faster development, and embed accessibility standards, thereby serving as an essential tool in the modern software engineering toolkit [7].

Incorporating WCAG 2.1 support directly into a Design System, rather than retrofitting individual web applications, presents a strategic advantage [8]. Technically, it ensures accessibility at the foundational level, promoting uniformity and reducing the potential for errors during application development. From a cost and effort perspective, it is economically more efficient, as it eliminates the need for repetitive, individual application assessments and modifications [9]. For project management, this centralized approach simplifies tracking and implementing accessibility standards, offering a clear roadmap for compliance and quality assurance across all digital assets.

IV. WCAG 2.1 Migration Procedure Of Design Systems

In this section, we delve into the systematic approach required for transitioning a Design System to comply with the Web Content Accessibility Guidelines (WCAG) 2.1. This process is not merely a technical upgrade but a strategic shift towards greater digital inclusivity. It involves a thorough evaluation of the existing system under WCAG 2.0 standards, identification of gaps, and the meticulous implementation of WCAG 2.1's enhanced criteria [10][11]. This section outlines the key steps in this migration process, emphasizing the pivotal role of web accessibility tools. These tools are instrumental in automating the assessment of compliance, identifying areas for improvement, and ensuring that the Design System not only meets but exceeds the requirements set forth in WCAG 2.1. By exploring the nuances of this migration, we aim to provide a comprehensive roadmap that assists developers, designers and stakeholders in effectively upgrading their systems for optimal accessibility and user experience [12].

A. *Evaluating Existing WCAG 2.0 Support in the Design System*

The initial step in migration is assessing the existing WCAG 2.0 accessibility support. Several web accessibility tools have been evaluated to study which WCAG 2.0 and WCAG 2.1 success criteria they support validation and their usage scenarios.

1. *Evaluating Keyboard Tab Focus*: Screen readers, such as JAWS, Windows Narrator, Apple Voiceover, NVDA and Google Talkback, are indispensable in ensuring seamless keyboard navigation [13]. As illustrated in [14, Fig. 1] and [14, Fig. 2], they primarily function to vocalize content on a screen for visually impaired users. However, they can also be essential tools for web developers and testers to evaluate and understand how a website or application behaves in terms of keyboard navigation and focus.

Fig. 1. Apple VoiceOver - Starting VoiceOver, Source [14]

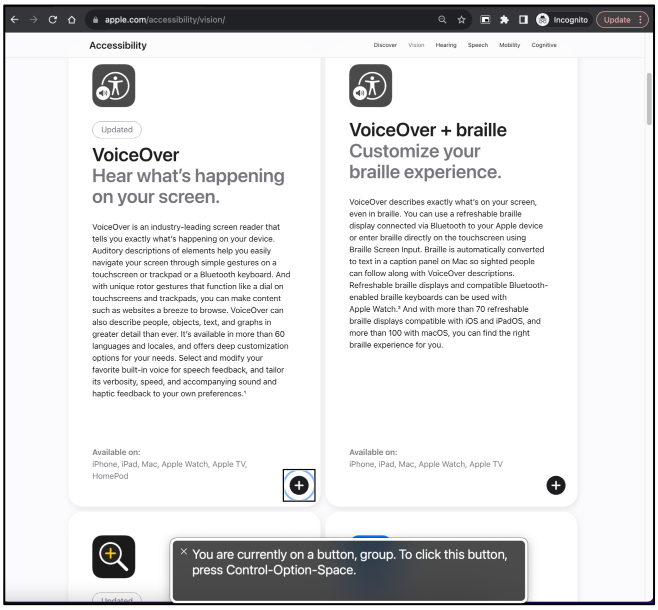

Fig. 2. Apple VoiceOver - Read aloud screen text, Source [14]

Here's how these screen readers assist with testing keyboard tab focus in a web application:

- *Keyboard Navigation Feedback*: When a user navigates through a web page using a keyboard (typically using the 'Tab' key to move forward and 'Shift + Tab' to move backward), screen readers vocalize the content or the name of the element that receives focus. This feedback helps testers verify that the focus order is logical and intuitive.

- *Visual Indicators*: While not a primary feature of screen readers, the interplay between screen readers and web browsers or operating systems often produces visual focus indicators (like a border or a highlighted outline) around the element that has keyboard focus. This visual cue helps developers and testers easily identify which element currently has focus.

- *Role and State Announcements*: When an interactive element (like a button or link) receives focus, the screen reader not only reads out its label or content but also its role (e.g., "button" or "link") and its current state (e.g., "checked" or "unchecked" for checkboxes) [15]. This feedback is invaluable for ensuring that all interactive elements are appropriately labeled and that their state is communicated effectively to users.

- *Detection of Focus Traps*: A focus trap occurs when a keyboard user can't navigate out of a particular section of a page or modal dialog, essentially getting "trapped." By using a screen reader while testing keyboard navigation, developers and testers can promptly identify and rectify such issues.

- *Skip Links*: Skip links are navigation aids that allow users to skip over repetitive content, like site navigation, to get to the main content more quickly. Screen readers announce these links when they receive focus, allowing testers to validate their presence and functionality.

- *ARIA Live Regions*: Sometimes, web applications have dynamic content that changes without a full page reload. ARIA (Accessible Rich Internet Applications) live regions are used to announce these changes. While this is not directly related to keyboard focus, navigating a site using a screen reader and keyboard can help testers ensure that important dynamic content changes are being announced as expected.

- *Error and Alert Announcements*: If there's an error or alert on the page, the screen reader should announce it, especially when it's related to a form input or action initiated by a keyboard user [16]. Testing with a screen reader can help ensure that these announcements are made correctly.

In conclusion, while the primary purpose of screen readers is to vocalize content for visually impaired users, their functionality is incredibly useful for developers and testers [17]. They provide crucial feedback about keyboard navigation, focus order, and the accessibility of interactive elements, ensuring that web applications are more usable and inclusive.

2. *HTML5 Markup Test*: Tools like Google Chrome Lighthouse, WAVE, aXe DevTools, and Accessibility Insights are pivotal in evaluating semantic structure and markup adherence.

- Google Chrome Lighthouse is an open-source, automated tool that assesses web pages across various categories, including accessibility. As illustrated in [18, Fig. 3] and [18, Fig. 4], Lighthouse provides a comprehensive audit of a page's elements, pointing out areas where semantic HTML elements (like `<header>`, `<nav>`, or `<main>`) should be used. It verifies whether the page’s code conforms to best-practices and highlights problems like a missed aria attribute or inappropriate elements hierarchies.

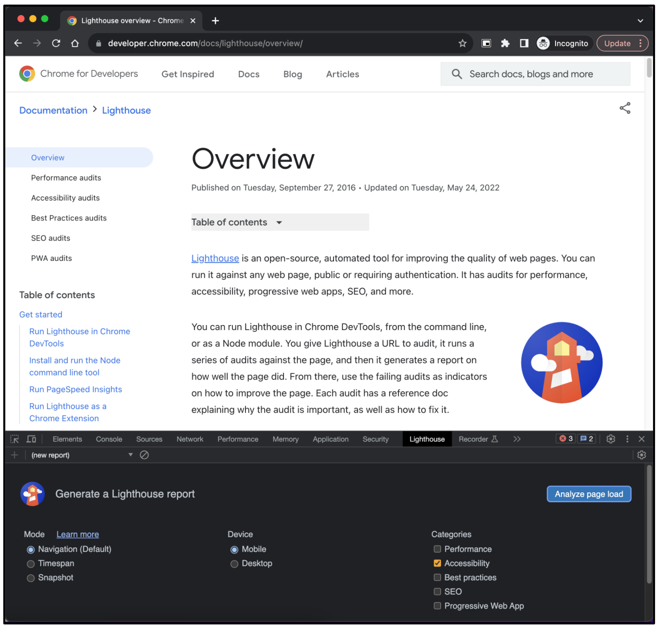

Fig. 3. Google Lighthouse - Running Accessibility Report, Source [18]

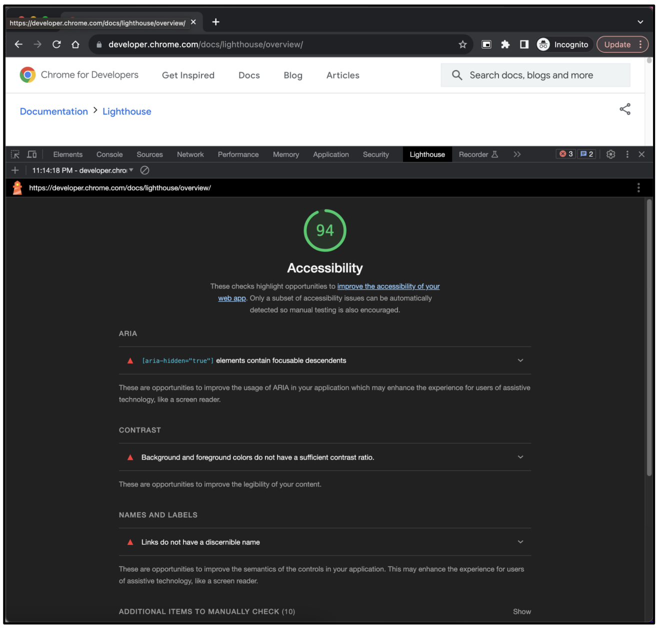

Fig. 4. Google Lighthouse - Report generated, Source [18]

- WAVE (Website Accessibility Evaluation Tool): It displays the icons or symbols in the page itself and therefore can give you an eye-pleasing view of whether your web content is accessible. WAVE visually points out errors, warnings, and content peculiarities like broken ALT attributes, wrong forms labeling, or misusing headers. Besides, this tool greatly facilitates revealing the structural errors as well as markup imperfections of one page.

- Accessibility Insights offers a suite of utilities to test, assess, and improve the accessibility of web applications. As illustrated in [20, Fig. 5] and [20, Fig. 6], Accessibility Insights, much like aXe (since it's built on the aXe core engine), evaluates the HTML structure, pointing out errors like missing role attributes, incorrect nesting, or skipped heading levels [19]. It presents its findings in an easy-to-understand format, often accompanied by visuals.

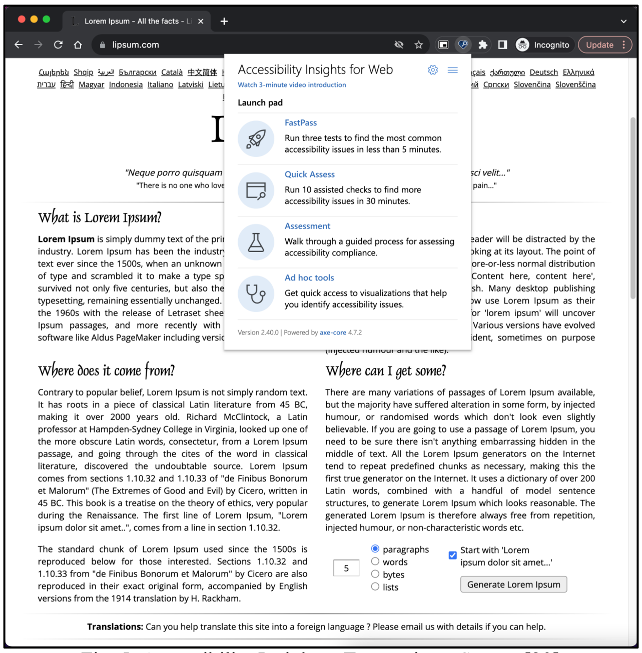

Fig. 5. Accessibility Insights - Test options, Source [20]

incorrect use of ARIA attributes, and elements that lack discernible text. Its thorough assessments are coupled with actionable guidance on how to rectify the issues, making it a favored tool among developers.

3. *Color Contrast Ratio Check*: The WebAIM Color Contrast checker plays a pivotal role in ensuring that content is readable and adheres to accessibility standards. As illustrated in [23, Fig. 7], it is specifically designed to assess the contrast between text and background colors to ensure that they meet accessibility standards. Here's a breakdown of how it assists with color contrast checks:

- *Immediate Feedback*: Users can input foreground and background colors using color pickers or by entering hexadecimal values [21]. The tool instantly provides feedback on whether the contrast ratio meets the required thresholds set by the WCAG (Web Content Accessibility Guidelines).

- *WCAG Compliance Levels*: This tool judges color schemes in accordance with either Level AA (WCAG 2.0) or AA (WCAG 2.1), providing more than minimum requirements for color contrasts. The app enables the developers as well as designs to know whether their colors conform to a certain level of accessibility.

- *Different Text Sizes*: Since a normal or a high text size may require different ranges for color contrasts, WebAIM's checker incorporates variable text-sizes for color contrasts determination. This is important since bigger texts are readable even in low contrast levels compared to smaller texts.

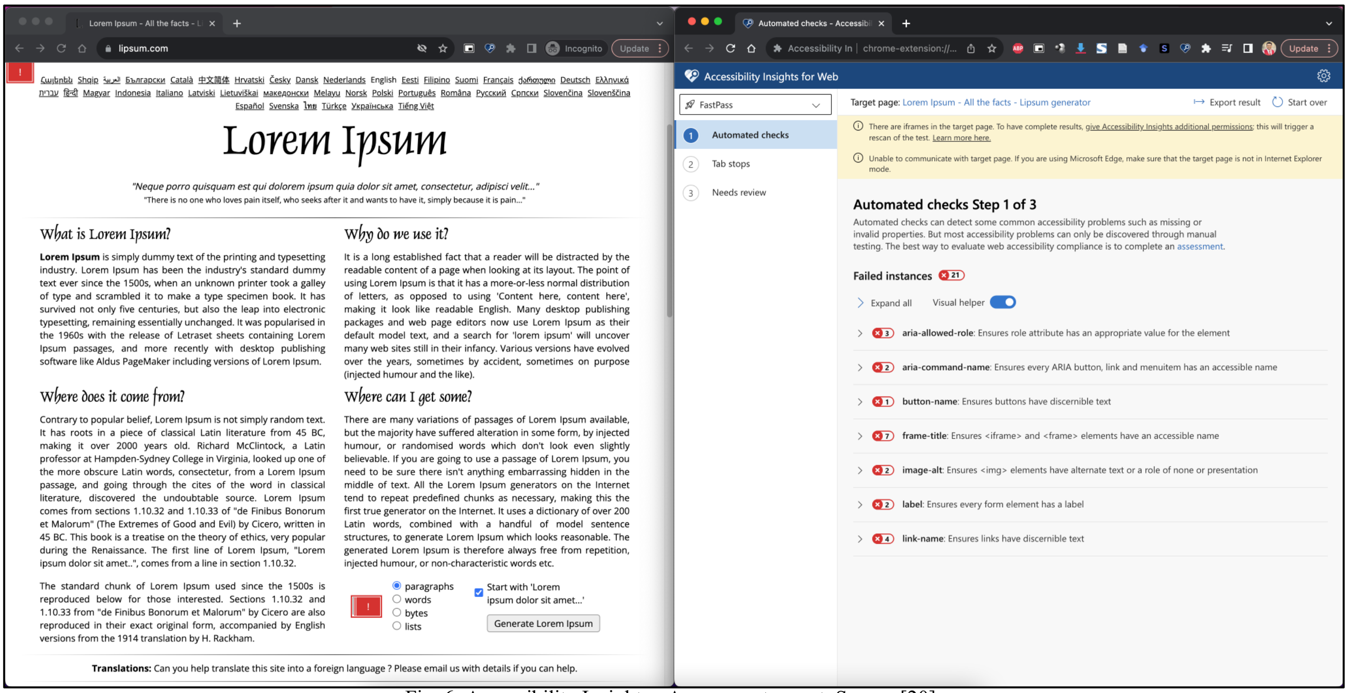

Fig. 6. Accessibility Insights - Assessment report, Source [20]

- aXe DevTools – allows testing for accessibility. Developers, and testers can find and fix defects in accessibility aspects. aXe offers a detailed analysis of markup issues, such as missing landmark roles,

- *Suggested Colors:* The tool also suggests foreground and background colors that will yield the recommended contrast ratios should a color combination not meet the minimums [22]. The designers need this feature because it allows them to get other equally good substitutes that meet the standards of accessibility.

- *Visual Representation*: Users can see a visual representation of the text with the chosen foreground and background colors, giving a realistic view of how text would appear against a particular background. This visualization can be particularly helpful for designers and developers to understand the real-world implications of their color choices.

- *Color Blindness Simulator*: Some versions of color contrast tools, including certain functionalities in WebAIM's offerings, simulate how content appears to users with specific types of color blindness. This additional perspective ensures that content remains accessible to a broader audience.

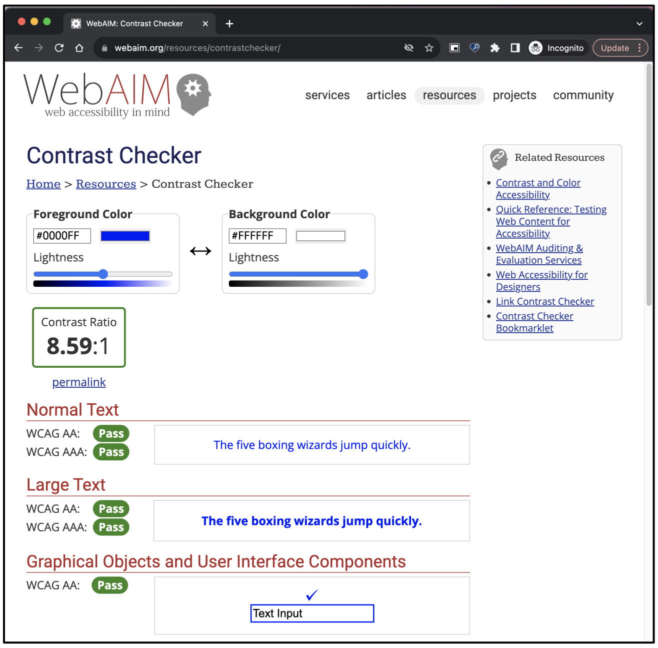

Fig. 7. WebAIM Color Contrast Checker - Contrast Ratio result, Source [23]

B. *Remediation of Defects in WCAG 2.0 Support*

Before transitioning to WCAG 2.1, it's vital to address and rectify any existing deficiencies in WCAG 2.0 support. Upon remediation, a re-evaluation using the tools mentioned ensures that the Design System is on the right track.

- *Prioritization*: As soon as deficiencies are identified, it becomes paramount to categorize them based on their severity and potential impact on users. Using a matrix that considers the complexity of the fix and the impact on user experience, issues like missing text alternatives for images, inoperable controls, or lack of keyboard accessibility often emerge as top priorities.

- *Component Refactoring*: This approach is particularly beneficial for recurring deficiencies. If a specific component, such as a button or a navigation bar, exhibits repeated issues, refactoring it at the source ensures all instances of its use across applications are rectified. Component-driven development, as advocated by scholars, emphasizes the importance of atomic design principles and the reusability of components for consistency and efficiency.

- *Inclusive Design Workshops*: Organizing collaborative workshops that involve designers, developers, and accessibility experts can foster a shared understanding [24]. These sessions can revolve around reimagining non-compliant components, understanding the importance of semantic HTML, and brainstorming innovative yet accessible design solutions.

- *Prototyping and User Testing*: Before finalizing the remediated components, it's beneficial to prototype them and test with a diverse user base. This iterative process, involving users, especially those using assistive technologies, can provide invaluable feedback on the effectiveness of the remediations.

- *Documentation and Training*: Updated documentation that captures the changes and the rationale behind them is essential. It serves as a reference for both current and future development teams. Additionally, hands-on training sessions, webinars, and workshops can further reinforce the importance of adhering to accessibility guidelines and the methodologies to achieve them.

C. *Diving into WCAG 2.1*

WCAG 2.1 introduces enhanced principles and success criteria, addressing the evolving needs of diverse users. However, migration isn't straightforward. The absence of an automated migration tool underscores the intricacy of the process. Successful WCAG 2.1 migration demands meticulous planning, time, and resources to ensure that the Design System embodies the new standards comprehensively [25]. Key among the WCAG 2.1 AA criteria is color contrast ratio, reflow, 400% zoom, and high contrast. In the journey to adapt a Design System to these guidelines, web accessibility tools play an indispensable role.

- *Color Contrast Ratio*: Tools like the WebAIM Contrast Checker validate foreground and background color combinations against recommended contrast ratios, ensuring that text is legible to users with color vision deficiencies.

- R*eflow*: Responsive design frameworks and tools, when combined with browser-based developer tools, help designers and developers ensure that content remains accessible and usable when displayed in a single column or when the viewport is resized

- *400% Zoom*: Web accessibility evaluation tools, such as axe-core, are adept at identifying issues that arise when content is zoomed to 400%, ensuring that the content does not lose information or functionality.

- *High Contrast*: Tools like Windows High Contrast Mode in combination with browser developer tools allow designers to check their designs in high contrast themes, ensuring that interfaces remain coherent and usable for those requiring heightened contrast.

Incorporating WCAG 2.1 support into a Design System necessitates a comprehensive understanding of these criteria and a suite of tools to validate conformance. By leveraging these tools, organizations can not only ensure compliance but foster a more inclusive digital environment.

D. Prioritizing Build and CI Automation

Post-migration, it becomes paramount to sustain and enhance the accessibility standards. Adopting the 'Shift Left' paradigm in software engineering ensures that accessibility considerations are embedded from the get-go. Implementing Build and CI automation through tools such as aXe for unit testing and CI, and Lighthouse CI, can proactively detect and prevent accessibility issues from permeating the codebase. aXe tools, known for their comprehensive accessibility checks, can be integrated directly into unit tests, running alongside regular test suites to ensure accessibility is considered at the smallest unit of the code [26]. Similarly, Lighthouse CI provides an automated set of checks that can be included in the CI pipeline, allowing for regular feedback on accessibility compliance with WCAG 2.1 guidelines during the build process.

By embedding these tools into the CI/CD pipeline, development teams can ensure continuous adherence to accessibility standards, allowing for immediate identification and remediation of issues, thereby significantly reducing the risk of non-compliance in the production environment. This approach not only facilitates a smoother transition to WCAG 2.1 but also embeds a culture of accessibility within the team's workflow.

V. Conclusion

The migration of a Design System to WCAG 2.1 is a complex yet essential endeavor that underscores an organization's commitment to inclusivity. The research presented demonstrates that web accessibility tools are invaluable in this process, offering automated and systematic approaches to evaluate and improve compliance. The findings suggest that these tools not only facilitate the identification and remediation of accessibility issues but also serve as a catalyst for embedding accessibility into the fabric of the development process [27]. By prioritizing Build and CI automation and adopting a 'Shift Left' approach, development teams can ensure continuous adherence to accessibility standards. Ultimately, the successful migration to WCAG 2.1 through the strategic use of web accessibility tools represents a significant step towards creating a universally accessible digital landscape.

Although previous studies have examined the use of accessibility tools in migration, this study is the first to concentrate on using these tools especially for the purpose of upgrading Design Systems to WCAG 2.1. Considering the crucial role that Design Systems play in contemporary web development, a thorough guide aimed at transferring these foundations might have significant effects on accessibility [28]. By offering a thorough tour of assessing WCAG 2.0 compliance in Design Systems, finding gaps, fixing bugs, putting improved WCAG 2.1 criteria into practice, and maintaining standards via CI/CD automation, this article fills a critical gap. Beyond what is already published in literature, the thorough examination of tools and their subtle uses throughout the migration process provides practical insights. Since Design Systems form the foundation of many web and mobile apps, our study helps firms to make sure these essential elements are in line with inclusive design principles and legally required standards. The practical tactics and tool suggestions in the paper provide a priceless resource for businesses dedicated to digital accessibility and inclusion.

References

[1] A. Utesheva, “Designing for Evolving Users,” in Designing Products for Evolving Digital Users: Study UX Behavior Patterns, Online Communities, and Future Digital Trends, A. Utesheva, Ed., Berkeley, CA: Apress, 2020, pp. 43–57. doi: 10.1007/978-1-4842-6379-2_3.

[2] S. Gesing et al., “A Vision for Science Gateways: Bridging the Gap and Broadening the Outreach,” in Practice and Experience in Advanced Research Computing, in PEARC ’21. New York, NY, USA: Association for Computing Machinery, Jul. 2021, pp. 1–8. doi: 10.1145/3437359.3465579.

[3] T. Coughlan and K. Lister, “The accessibility of administrative processes: Assessing the impacts on students in higher education,” in Proceedings of the 15th International Web for All Conference, in W4A ’18. New York, NY, USA: Association for Computing Machinery, Apr. 2018, pp. 1–10. doi: 10.1145/3192714.3192820.

[4] L. Pettit, Cultivating a Digital Culture for Effective Patient Engagement: A Strategic Framework and Toolkit for Health-Provider Websites. Boca Raton: Productivity Press, 2020. doi: 10.4324/9780429399657.

[5] Narayan Palani, Automated Software Testing with Cypress. Auerbach Publications. Accessed: Dec. 09, 2023. [Online]. Available: https://www.taylorfrancis.com/chapters/mono/10.1201/9781003145110-16/cypress-based-api-testing-narayan-palani

[6] J. Pelzetter, “Using Ontologies as a Foundation for Web Accessibility Tools,” in Proceedings of the 15th International Web for All Conference, in W4A ’18. New York, NY, USA: Association for Computing Machinery, Apr. 2018, pp. 1–2. doi: 10.1145/3192714.3196316.

[7] H. Shah, “Harnessing customized built-in elements -- Empowering Component-Based Software Engineering and Design Systems with HTML5 Web Components,” in Computer Science & IT Conference Proceedings, in 22, vol. 13. Academy & Industry Research Collaboration Center, Nov. 2023, pp. 247–259. doi: 10.5121/csit.2023.132219.

[8] R. Davies, Navigating Telehealth for Speech and Language Therapists: The Remotely Possible in 50 Key Points. London: Routledge, 2022. doi: 10.4324/9781003269724.

[9] P. L. M. Shea Peter, Ed., Transforming Digital Learning and Assessment: A Guide to Available and Emerging Practices and Building Institutional Consensus. New York: Routledge, 2023. doi: 10.4324/9781003448334.

[10] H. Shah, “Advancing Web Accessibility -- A guide to transitioning Design Systems from WCAG 2.0 to WCAG 2.1,” in Computer Science & IT Conference Proceedings, in 22, vol. 13. Academy & Industry Research Collaboration Center, Nov. 2023, pp. 233–245. doi: 10.5121/csit.2023.132218.

[11] P. L. Gorski, E. von Zezschwitz, L. Lo Iacono, and M. Smith, “On providing systematized access to consolidated principles, guidelines and patterns for usable security research and development†,” Journal of Cybersecurity, vol. 5, no. 1, p. tyz014, Jan. 2019, doi: 10.1093/cybsec/tyz014.

[12] J. Oppenlaender, T. Tiropanis, and S. Hosio, “CrowdUI: Supporting Web Design with the Crowd,” Proc. ACM Hum.-Comput. Interact., vol. 4, no. EICS, p. 76:1-76:28, Jun. 2020, doi: 10.1145/3394978.

[13] J. Radianti and T. Gjøsæter, “Digital Volunteers in Disaster Response: Accessibility Challenges,” in Universal Access in Human-Computer Interaction. Multimodality and Assistive Environments, M. Antona and C. Stephanidis, Eds., in Lecture Notes in Computer Science. Cham: Springer International Publishing, 2019, pp. 523–537. doi: 10.1007/978-3-030-23563-5_42.

[14] "Apple - Accessibility - Vision," Apple. Accessed: Nov. 19, 2023. Available: https://www.apple.com/accessibility/vision/
[15] K.-L. Lubin, "Conversations Towards Practiced AI – HCI Heuristics," in HCI International 2022 – Late Breaking Papers: Interacting with eXtended Reality and Artificial Intelligence, J. Y. C. Chen, G. Fragomeni, H. Degen, and S. Ntoa, Eds., in Lecture Notes in Computer Science. Cham: Springer Nature Switzerland, 2022, pp. 377–390. doi: 10.1007/978-3-031-21707-4_27.
[16] G. Praveen Raj Kumar; Yadav, Chandra Shekhar; Vishnoi, Shweta; Singh, Laxman; Pachauri, "EBSCOhost | 153365075 | Team Cognition Approach in Agile Software Development.," vol. 14, no. 4, pp. 18–25, 2021, doi: 10.25103/jestr.144.03.
[17] S. Burgstahler, "Designing Inclusive Formal and Informal Online Learning: What Do Instructors Need to Know?," in Guide to Digital Accessibility, Routledge, 2023.
[18] "Google Lighthouse overview," Chrome for Developers. Accessed: Nov. 19, 2023. Available: https://developer.chrome.com/docs/lighthouse/overview/
[19] F. Trevisan, "Beyond accessibility: Exploring digital inclusivity in US progressive politics," New Media & Society, vol. 24, no. 2, pp. 496–513, Feb. 2022, doi: 10.1177/14614448211063187.
[20] "Accessibility Insights." Accessed: Oct. 29, 2023. Available: https://accessibilityinsights.io
[21] L. Seixas Pereira, J. Coelho, A. Rodrigues, J. Guerreiro, T. Guerreiro, and C. Duarte, "Authoring accessible media content on social networks," in Proceedings of the 24th International ACM SIGACCESS Conference on Computers and Accessibility, in ASSETS '22. New York, NY, USA: Association for Computing Machinery, Oct. 2022, pp. 1–11. doi: 10.1145/3517428.3544882.
[22] A. Daniel, "Project-Based Learning as a Means of Full Inclusion to Students With Special Needs," Advances in educational marketing, administration, and leadership book series, pp. 314–333, Oct. 2023, doi: 10.4018/978-1-6684-8737-2.ch015.
[23] WebAIM, "WebAIM Contrast Checker." Accessed: Sep. 21, 2023. Available: https://webaim.org/resources/contrastchecker/
[24] "Arduino-Based Control and Monitoring System of Household Appliances Status using Android Application," International Journal of Engineering and Advanced Technology, vol. 9, no. 3, pp. 3873–3879, Feb. 2020, doi: 10.35940/ijeat.c6377.029320.
[25] Tomás Murillo-Morales, P. Heumader, and Klaus Miesenberger, "Automatic Assistance to Cognitive Disabled Web Users via Reinforcement Learning on the Browser," Lecture Notes in Computer Science, pp. 61–72, Jan. 2020, doi: 10.1007/978-3-030-58805-2_8.
[26] D. Lux, "A Framework for Citizen-Centric Government Websites," IGI Global eBooks, pp. 20–34, Aug. 2010, doi: 10.4018/978-1-61692-018-0.ch002.
[27] L. F. Londoño-Rojas, V. Tabares-Morales, and N. Duque-Méndez, "AET tool for hybrid accessibility evaluation," Univers. Access Inf. Soc., vol. 22, no. 2, pp. 655–661, Oct. 2021, doi: 10.1007/s10209-021-00846-8.
[28] G. Brajnik, "Comparing accessibility evaluation tools: a method for tool effectiveness," *Universal Access in the Information Society*, vol. 3, pp. 252–263, Oct. 2004, doi: 10.1007/s10209-004-0105-y.